Control y Automatización de una Grúa Portacontenedores Portuaria (STS)

Proyecto Global Integrador - Autómatas y Control Discreto

Peña Lautaro (13099) Peralta Bruno (13220)

2026

Ing. Mecatrónica — UNCuyo

Contenido

1. Introducción

2. La planta (provista)

3. Nivel 2 — Control regulatorio

4. Nivel 1 — Control supervisor (DEDS)

5. Nivel 0 — Protección

6. Comunicación y Animación con Python

7. Implementación en CODESYS

8. Generación automática de código ST

9. Conclusiones

Introducción

El sistema: grúa portacontenedores STS

- Grúa de muelle tipo pórtico (*Ship-to-Shore*) para carga/descarga de contenedores en barcos.
- Dos movimientos principales, simultáneos, en un plano vertical (2D):
  - **Carro** (*Trolley*): traslación horizontal.
  - **Izaje** (*Hoist*): subida/bajada de la carga.
- Carga suspendida que se **balancea como un péndulo** $\Rightarrow$ ejes acoplados.
- Objetivo: **control de accionamientos** y **operación semi-automática coordinada**.



Arquitectura de control jerárquica

Sistema híbrido de control y protección — 3 niveles

Nivel	Rol	Tipo / T_s
2	Regulatorio (torque/velocidad)	Continuo muestreado — 1 ms
1	Supervisor (modos, coordinación)	**Eventos discretos** — 20 ms
0	Protección (seguridad)	Eventos discretos — 20 ms

- Jerarquía creciente: el Nivel 0 **sobrescribe** a los superiores ante falla.

La planta (provista)

Modelo dinámico híbrido de la planta

La cátedra entrega el modelo completo. Es **híbrido**: combina dinámicas continuas con cambios discretos de estado.

Variables continuas (flow)

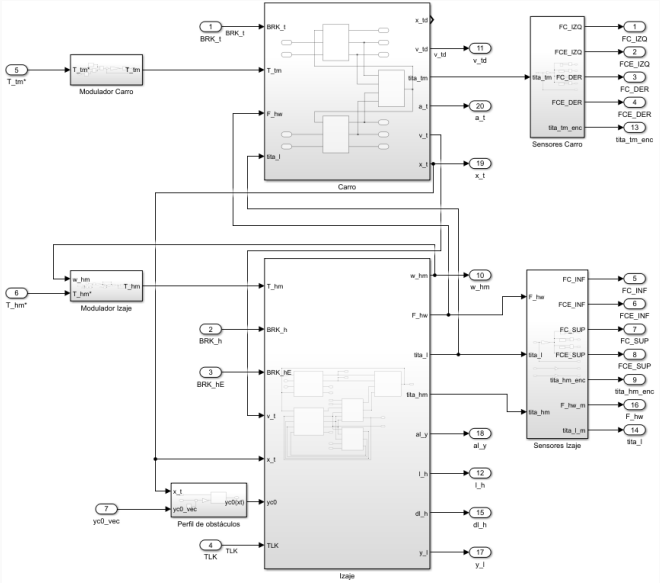
- Posición/velocidad de carro x_t, v_t e izaje y_h, v_h.
- Ángulo de balanceo θ_l, tensión de cable F_{hw}.
- Torques, fuerzas de contacto.

Estados discretos (jump)

- Twistlocks TLK: abierto/cerrado.
- Contacto: suspendido/apoyado.
- Cable: tenso/flojo.
- Frenos NC *BRK*: abierto/cerrado.

Carro e izaje acoplados por la carga pendular; interacción con el perfil de obstáculos del entorno.

Implementación de la planta en Simulink



Parámetros (dados)

M_t	30 000 kg
M_s	15 000 kg
$M_{c\,\text{máx}}$	50 000 kg
i_t / i_h	30/22
$v_{t\,\text{máx}}$	4 m/s
v_h	1,5–3 m/s

Sensores: encoders, fines de carrera, celda de carga, ángulo de spreader, LIDAR.

Subsistema del carro: visión general

Función del sistema

Traslación horizontal del carro mediante un **motor** que acciona un **tambor**, el cual tira del carro a través de cables de acero.

Modelo: sistema electromecánico de 2 masas

- **Masa rotacional**: inercia del accionamiento (motor + tambor + reductor).
- **Masa traslacional**: el carro M_t sobre los rieles.
- Acopladas por un **cable elástico** (resorte-amortiguador).

Hipótesis

Transmisión rígida y **1 cable equivalente** de carro, siempre tensado, con elasticidad longitudinal amortiguada.

Dinámica del carro (2.ª ley de Newton)

Movimiento horizontal del carro

$$M_t \frac{dv_t(t)}{dt} = F_{tw}(t) - b_t\, v_t(t) + 2F_{hw}(t)\sin\theta_l(t)$$

Términos

- $F_{tw}(t)$: fuerza de tracción del cable del carro.
- $b_t\, v_t(t)$: fricción viscosa sobre los rieles.
- $2F_{hw}(t)\sin\theta_l(t)$: fuerza horizontal por el balanceo de la carga.

Valores: $M_t = 30\,000$ kg ; $b_t = 90$ Ns/m.

Acoplamiento con el izaje

El término $2F_{hw}\sin\theta_l$ es la **perturbación del balanceo**: conecta la dinámica del carro con la de la carga suspendida.

Acoplamiento elástico del cable

Modelo resorte-amortiguador

El cable transmite la fuerza entre el tambor y el carro:

$$F_{tw}(t) = K_{tw}\left(x_{td}(t) - x_t(t)\right) + b_{tw}\left(v_{td}(t) - v_t(t)\right)$$

Parámetros y variables

- $K_{tw} = 480 \times 10^3$ N/m (rigidez del cable).
- $b_{tw} = 3 \times 10^3$ Ns/m (amortiguamiento).
- x_{td}, v_{td}: posición y velocidad lineal **equivalente del tambor** (en su superficie).

Interpretación

La fuerza depende de la **diferencia** tambor–carro: el cable se estira y empuja/tira según el desfasaje entre ambos.

Dinámica del accionamiento: dos ejes

Eje lento (tambor)

$$J_{td}\,\dot{\omega}_{td}(t) = T_{td}(t) - b_{td}\,\omega_{td}(t) - T_{tdl}(t)$$

- $J_{td} = 1200$ kgm²
- $b_{td} = 1{,}8$ Nms/rad
- $r_{td} = 0{,}5$ m
- $T_{tdl}(t) = F_{tw}(t)\,r_{td}$ (carga del cable)

Eje rápido (motor)

$$J_{tm+tb}\,\dot{\omega}_{tm}(t) = T_{tm}(t) + T_{tb}(t, BRK_t) - b_{tm}\,\omega_{tm}(t) -$$

- $J_{tm+tb} = 7$ kgm²
- $b_{tm} = 6$ Nms/rad
- $T_{tm}(t)$: torque del motor
- $T_{tb}(t, BRK_t)$: torque del freno

Unión de los ejes

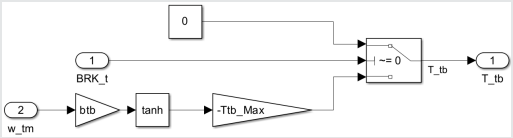
Reductor de relación $i_t = 30$: $\omega_{td}\,i_t = \omega_{tm}$ y $T_{td} = i_t\,T_{tml}$.

Freno mecánico (Normal Cerrado)

Torque de frenado — depende de BRK_t

$$T_{tb}(BRK_t) = \begin{cases} 0 & BRK_t = 1 \text{ (abierto)} \\ -T_{tb\,\text{máx}} \tanh(b_{tb}\,\omega_{tm}) & BRK_t = 0 \text{ (cerrado)} \end{cases}$$

- $T_{tb\,\text{máx}} = 5 \times 10^3$ Nm
- $b_{tb} = 5 \times 10^6$ Nms/rad



Por qué tanh

Función continua y abrupta: frena fuerte pero evita la **discontinuidad numérica** del signo en la simulación.

Modelo equivalente referido al motor

Reducción a un solo eje

Se refiere la dinámica del tambor al eje del motor usando $i_t = 30$, agrupando inercias y fricciones:

$$J_{t,eq} = J_{tm+tb} + \frac{J_{td}}{i_t^2} = 7 + \frac{1200}{30^2} \approx 8{,}33 \text{ kgm}^2$$

$$b_{t,eq} = b_{tm} + \frac{b_{td}}{i_t^2} = 6 + \frac{1{,}8}{30^2} \approx 6{,}002 \text{ Nms/rad}$$

Resultado

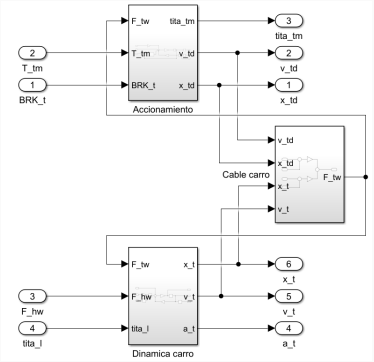
Ecuación de estado única para todo el accionamiento del carro:

$$\frac{d\omega_{tm}}{dt}(t) = \frac{1}{J_{t,eq}} \left[T_{tm}(t) + T_{tb}(t, BRK_t) - b_{t,eq}\,\omega_{tm}(t) - \frac{r_{td}}{i_t} F_{tw}(t) \right]$$

Modelo dinámico completo del carro

Ecuaciones de estado acopladas

$$
\begin{cases}
\dot{\omega}_{tm}(t) = \frac{1}{J_{t,eq}}\left[T_{tm}(t) + T_{tb}(t, BRK_t) - b_{t,eq}\,\omega_{tm}(t) - \frac{r_{td}}{i_t}F_{tw}(t)\right] \\
\dot{v}_t(t) = \frac{1}{M_t}\left[F_{tw}(t) - b_t\,v_t(t) + 2F_{hw}(t)\sin\theta_l(t)\right] \\
\dot{x}_t(t) = v_t(t) \\
F_{tw}(t) = K_{tw}\big(x_{td}(t) - x_t(t)\big) + b_{tw}\big(v_{td}(t) - v_t(t)\big)
\end{cases}
$$



Variables auxiliares

$$
v_{td}(t) = \frac{r_{td}}{i_t}\,\omega_{tm}(t) \quad ; \quad x_{td}(t) = \int v_{td}(t)\,dt
$$

Dimensionalidad

3 estados continuos (ω_{tm}, v_t, x_t) + el **freno** como evento discreto (BRK_t).

Subsistema de izaje

Características del sistema

Controla la elevación de la carga mediante:

- Motor eléctrico accionando un **tambor**
- Sistema de cableado con relación de transmisión **2:1** (aparejo móvil)
- Carga suspendida: **2 grados de libertad**

Característica clave:

- Dinámica híbrida

Parámetros discretos

Masa total (m_l)

$$m_l(\text{TLK}) = \begin{cases} M_s & \text{si TLK} = 0 \text{ (solo spreader)} \\ M_s + M_{cX} & \text{si TLK} = 1 \text{ (spreader + contenedor)} \end{cases}$$

Altura de la carga (h_c)

$$h_c(\text{TLK}) = \begin{cases} 0 & \text{si TLK} = 0 \\ H_c & \text{si TLK} = 1 \end{cases}$$

Ecuaciones de movimiento de la carga

Dinámicas acopladas

$$m_l(TLK)\,\frac{d\,v_{lx}(t)}{dt} = -2F_{hw}(t, l(t) - l_h(t))\,\sin\theta_l(t) + F_{cx}(t, \delta y)$$

$$m_l(TLK)\,\frac{d\,v_{ly}(t)}{dt} = 2F_{hw}(t, l(t) - l_h(t))\,\cos\theta_l(t) - m_l(TLK)\,g + F_{cy}(t, \delta y)$$

$$\delta y = y_l(t) - h_c(TLK) - y_{c0}(x_l(t), t)$$

Interacción con el entorno

Condición de contacto

Cuando: $\delta y \leq 0$

Fuerza vertical

$$F_{cy} = \begin{cases} K_{cy}(y_{c0} - (y_l - h_c)) - b_{cy} v_{ly} & \text{(Apoyado)} \\ 0 & \text{(Suspendido)} \end{cases}$$

Fuerza horizontal

$$F_{cx} = \begin{cases} -b_{cx} v_{lx} & \text{(Apoyado)} \\ 0 & \text{(Suspendido)} \end{cases}$$

Acoplamiento elástico del cable

Resorte-amortiguador con rigidez variable

La rigidez y amortiguamiento escalan con la longitud del cable bajo tensión:

$$K_{hw}(l_h) = \frac{k_{hwu}}{2l_h + L_{h0}} \quad ; \quad b_{hw}(l_h) = b_{hwu}(2l_h + L_{h0})$$

Fuerza de tensión

$$F_{hw} = \begin{cases} K_{hw}(l_h) \cdot 2(l - l_h) + b_{hw}(l_h) \cdot 2(\dot{l} - \dot{l}_h) & \text{(Tenso)} \\ 0 & \text{(Flojo)} \end{cases}$$

Dinámica del accionamiento

Ecuaciones de los dos ejes

- **Eje lento(tambor):**

$$J_{hd+hEb}\,\frac{d\,\omega_{hd}(t)}{dt} = T_{hd}(t) + T_{hEb}(t, BRK_{hE}) - b_{hd}\,\omega_{hd}(t) - T_{hdl}(t)$$

- **Eje rápido(motor):**

$$J_{hm+hb}\,\frac{d\,\omega_{hm}(t)}{dt} = T_{hm}(t) + T_{hb}(t, BRK_{h}) - b_{hm}\,\omega_{hm}(t) - T_{hml}(t)$$

Frenos mecánicos

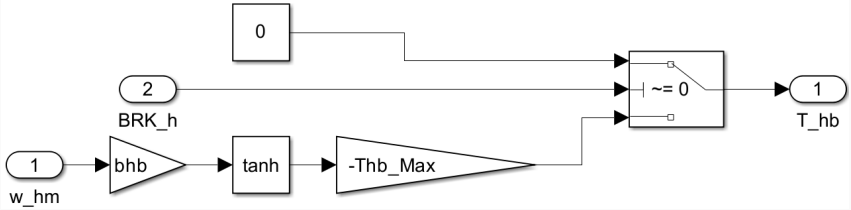


Figura 1: Freno mecánico de operación izaje

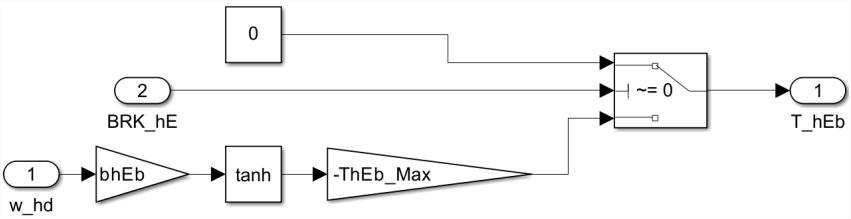


Figura 2: Freno mecánico de emergencia izaje

Modelo equivalente referido al motor

Reducción eje equivalente

Al igual que el carro, se refiere la dinámica del tambor al motor usando i_h:

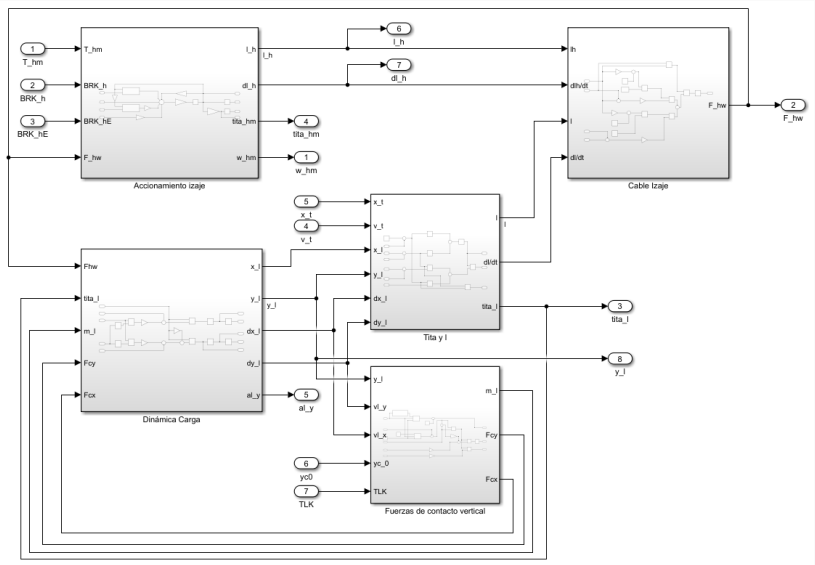
$$J_{h,eq} = J_{hm+hb} + \frac{J_{hd+hEb}}{i_h^2}$$

$$b_{h,eq} = b_{hm} + \frac{b_{hd}}{i_h^2}$$

Variables auxiliares

- Longitud geométrica: $l(t) = \sqrt{(x_l - x_t)^2 + (Y_{t0} - y_l)^2}$
- Velocidad de cambio: $\frac{dl}{dt} = \frac{(x_l - x_t)(v_{lx} - v_t) - (Y_{t0} - y_l)v_{ly}}{l}$
- Ángulo de balanceo: $\theta_l = \text{atan2}\left(\frac{x_l - x_t}{Y_{t0} - y_l}\right)$

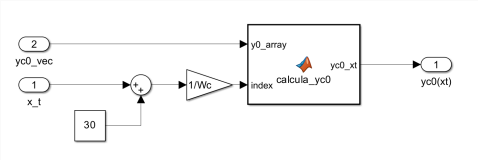
Subsistema izaje completo



Discretización del perfil de obstáculos

Relevamiento por LIDAR

El LIDAR del carro releva el **perfil de obstáculos** $y_{c0}(x, t)$ (pila de contenedores, muelle, barco).



Discretización por ancho de contenedor

Paso $= W_c = 2{,}44$ m (contenedor ISO):

$$N_{x_t} = \left\lfloor \frac{x_{t\,\text{máx}} - x_{t\,\text{mín}}}{W_c} \right\rfloor + 1 = \left\lfloor \frac{80}{2{,}44} \right\rfloor + 1 = 33$$

Índices $i = 1, \ldots, 33$.

Vector de perfil $y_{c0\,vec}$

Vector de 33 alturas conocido por el autómata; se **actualiza** al mover contenedores. El modo automático exige el perfil **escaneado por completo**.

Sensores

Sensor carro

- **Encoder:** Mide la posición angular del motor del carro $\theta_{tm}(t)$

$$x_t(t) = \theta_{tm}(t)\,\frac{r_{tm}}{i_t}$$

Sensores izaje

- **Encoder:** Mide posición angular del motor de izaje $\theta_{hm}(t)$

$$y_h(t) = \theta_{hm}(t)\,\frac{r_{hd}}{2\,i_h}$$

- **Celda de carga:** Mide fuerza de tensión $F_{hw}(t)$ en el cable de izaje
- **Sensor de ángulo de spreader:** Mide ángulo de balanceo $\theta_l(t)$ y velocidad angular $\dot{\theta}_l(t)$ de la carga suspendida

Fines de carrera

FC del carro

$$\mathrm{FC}_{t,\mathrm{der}} = \begin{cases} 0 & \text{si } x_t < x_{t\,\mathrm{máx}} \\ 1 & \text{si } x_t \geq x_{t\,\mathrm{máx}} \\ 1 & \text{si } x_t \geq x_{t\,\mathrm{máx}} + 0{,}5 \text{ m (E)} \end{cases}$$

$$\mathrm{FC}_{t,\mathrm{izq}} = \begin{cases} 0 & \text{si } x_t > x_{t\,\mathrm{mín}} \\ 1 & \text{si } x_t \leq x_{t\,\mathrm{mín}} \\ 1 & \text{si } x_t \leq x_{t\,\mathrm{mín}} + 0{,}5 \text{ m (E)} \end{cases}$$

FC del izaje

$$\mathrm{FC}_{h,\mathrm{sup}} = \begin{cases} 0 & \text{si } y_h < y_{h\,\mathrm{máx}} \\ 1 & \text{si } y_h \geq y_{h\,\mathrm{máx}} \\ 1 & \text{si } y_h \geq y_{h\,\mathrm{máx}} + 0{,}5 \text{ m (E)} \end{cases}$$

$$\mathrm{FC}_{h,\mathrm{inf}} = \begin{cases} 0 & \text{si } y_h > y_{h\,\mathrm{mín}} \\ 1 & \text{si } y_h \leq y_{h\,\mathrm{mín}} \\ 1 & \text{si } y_h \leq y_{h\,\mathrm{mín}} + 0{,}5 \text{ m (E)} \end{cases}$$

Modelo sensores

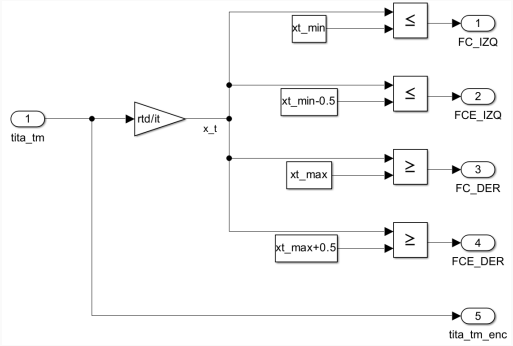
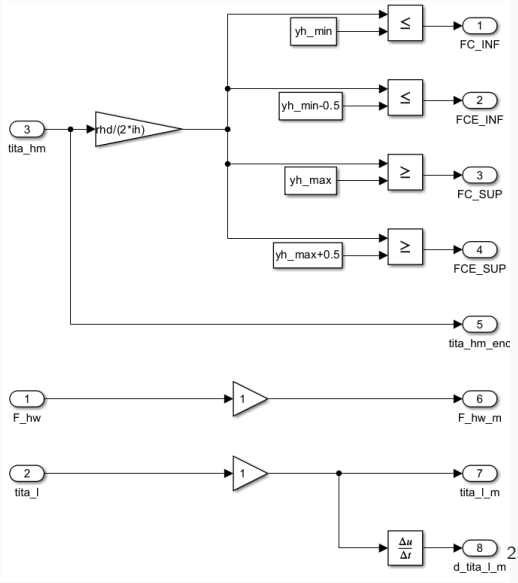


Figura 3: Diagrama de bloques sensores carro e izaje Simulink



Moduladores de torque

Función en la simulación

Modelan las limitaciones de los **accionamientos reales** (motores eléctricos)

- No es posible aplicar torque instantáneo como una fuerza mecánica ideal
- Los motores reales tienen capacidad limitada y retardo dinámico

Dos funciones clave

1. **Saturación**: limita torque a máximo permitido $T_{\text{máx}}$
2. **Filtrado**: modela retardo electromagnético con constante $\tau = 1$ ms

Ecuación diferencial

Vincula el torque limitado comandado $T^*_{\text{Lim}}(t)$ con el torque físico real $T(t)$:

$$\tau \frac{dT(t)}{dt} = T^*_{\text{Lim}}(t) - T(t)$$

Modulador del carro

Ecuación

$$T^*_{tm\ Lim}(k\ T_{s2} \leq t < (k+1)\ T_{s2}) = \min\left(+T_{tm\,\text{máx}}, \max\left(-T_{tm\,\text{máx}}, T^*_{tm}(k\ T_{s2})\right)\right)$$

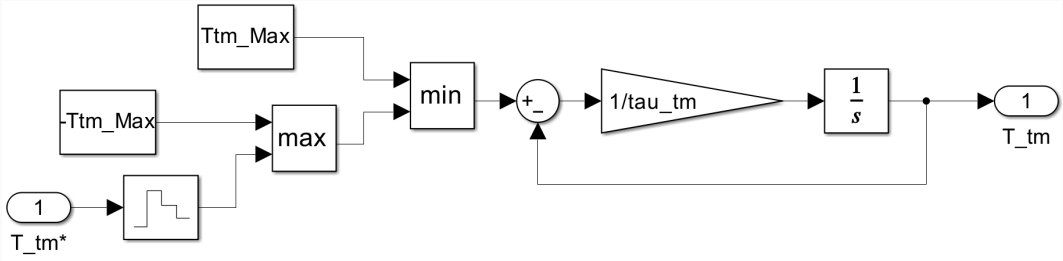


Figura 4: Diagrama de bloques del modulador de torque del carro

Modulador de torque del izaje

Límite de torque variable con la velocidad

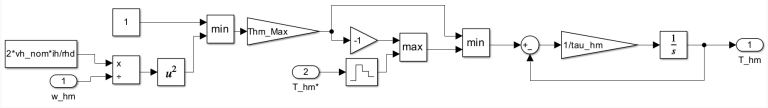
A diferencia del carro, el torque máximo se **reduce a alta velocidad** (**debilitamiento de campo**, potencia constante):

$$T_{hm\,\text{máx}}(t) = T_{hm\,MAX}\ \text{mín}\left(1, \left(\frac{\omega_{hm\,RATED}}{\omega_{hm}(t)}\right)^2\right)$$

$$T_{hm\,MAX} = 20 \times 10^3\ \text{Nm} \quad ; \quad \omega_{hm\,RATED} = \frac{2\,v_{h\,nom}\,i_h}{r_{hd}} = 88\ \text{rad/s}$$

Saturación y filtro

$T^*_{hm\,Lim}(k) = \text{sat}\big(T^*_{hm}(k),\ \pm T_{hm\,\text{máx}}(t)\big)$; filtro de 1.er orden ($\tau = 1$ ms) y ZOH, igual que el carro.



Nivel 2 — Control regulatorio

Nivel 2

Función del control regulatorio

Genera comandos de **torque** T_{tm}^* a partir de referencias de **velocidad** v_t^* provenientes del Nivel 1 (supervisor)

Objetivos de control

- Seguimiento preciso de referencias de velocidad
- Suavidad en la respuesta transitoria
- Atenuación de perturbaciones: balanceo de carga, variaciones de masa
- Operación en tiempo discretizado (muestreado)

Modelo del carro para control de velocidad

Dinámica simplificada de traslación

Se desprecian dinámicas rotacionales internas del accionamiento y se agrupan perturbaciones externas:

$$M_t \frac{dv_t(t)}{dt} + b_t\, v_t(t) = F_{\text{cmd}}(t) + F_{\text{pert}}(t)$$

Términos de la ecuación

- $F_{\text{cmd}}(t) = K_f\, T_{tm}(t)$ con $K_f = \frac{i_t}{r_{td}}$
- $F_{\text{pert}}(t) = 2F_{hw}(t)\sin(\theta_l(t))$

Transformada de Laplace

$$G_{vel,t}(s) = \frac{V_t(s)}{T_{tm}(s)} = \frac{K_f}{M_t\, s + b_t} \approx \frac{K_f}{M_t\, s}$$

Controlador PI

Tipo de controlador

Proporcional-integral (PI):

$$C_{\text{vel}}(s) = K_p + \frac{K_i}{s}$$

Función de transferencia a lazo cerrado

$$T(s) = \frac{C_{\text{vel}}(s)\, G_{vel,t}(s)}{1 + C_{\text{vel}}(s)\, G_{vel,t}(s)} = \frac{K_p K_f\, s + K_i K_f}{M_t\, s^2 + K_p K_f\, s + K_i K_f}$$

Asignación de polos: ecuación característica

Polinomio característico deseado de 2do orden

$$s^2 + 2\zeta\omega_n s + \omega_n^2$$

con amortiguamiento ζ y frecuencia natural ω_n

Comparación e igualación de coeficientes

$$\frac{K_p K_f}{M_t} = 2\zeta\omega_n \quad \Rightarrow \quad K_p = \frac{2\zeta\omega_n M_t}{K_f}$$

$$\frac{K_i K_f}{M_t} = \omega_n^2 \quad \Rightarrow \quad K_i = \frac{\omega_n^2 M_t}{K_f}$$

Parámetros elegidos

- $\omega_n = 0{,}8$ rad/s
- $\zeta = 1$

Modelo controlador PI del carro

Comando de torque en tiempo discreto

$$T^*_{tm}(k) = K_p\, e_v(k) + K_i \sum_{j=1}^{k} \frac{T_s}{2} \left[e_v(j) + e_v(j-1) \right], \quad \textit{siendo} \quad e_v(k) = v_t^* - v_t(k)$$

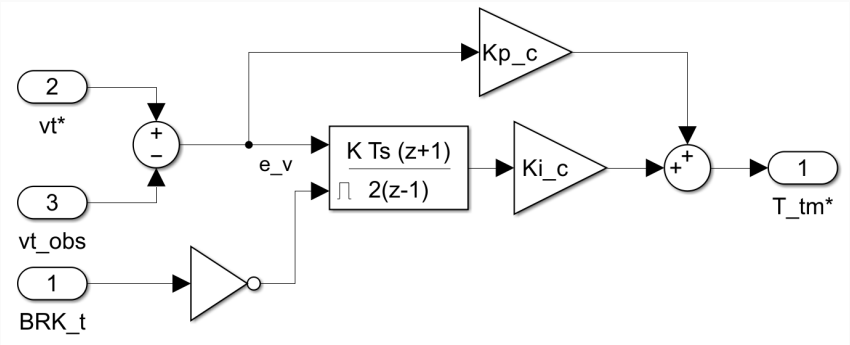


Figura 5: Diagrama de bloques del controlador PI discreto del carro en Simulink

Control de balanceo (Anti-Sway): estrategia

Control desacoplado (no basado en el modelo completo)

Se aprovechan dos dinámicas con escalas de tiempo **muy separadas**:

- **Accionamiento del carro**: rápido (establecimiento < 100 ms).
- **Péndulo de la carga**: lento (periodo natural 2–12 s, según l_h).

Hipótesis de desacople

Con el lazo de velocidad bien sintonizado, $v_t(t) \approx v_t^*(t)$: la dinámica del actuador se separa de la pendular.

Idea

El balanceo se amortigua **modulando la velocidad de referencia** del carro, no con un actuador extra.

Deducción de la ley de control no lineal

Péndulo excitado por la aceleración del carro

$$\ddot{\theta}_l = -\frac{g}{l_h}\sin\theta_l - \frac{\ddot{x}_t}{l_h}\cos\theta_l$$

Ley antibalanceo proporcional al ángulo

$v_t^* = v_{t\,target} + v_{sway}$, con $v_{sway} = K_{sway}\,\theta_l \quad \Rightarrow \quad \ddot{x}_t \approx \dot{v}_{t\,target} + K_{sway}\,\dot{\theta}_l$.

$$\ddot{\theta}_l + \underbrace{\left(\frac{K_{sway}}{l_h}\cos\theta_l\right)}_{\text{amortiguamiento}}\dot{\theta}_l + \frac{g}{l_h}\sin\theta_l = -\frac{\dot{v}_{t\,target}}{l_h}\cos\theta_l$$

Resultado clave

La velocidad inyectada se vuelve un **amortiguamiento viscoso no lineal**. Como $\cos(\theta_l) > 0$, es estable a cualquier ángulo operativo **sin** aproximar $\sin\theta_l \approx \theta_l$.

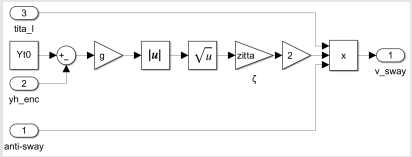
Sintonía por linealización local (gain scheduling)

Dinámica local ($\theta_l \approx 0$)

$$\ddot{\theta}_l + \frac{K_{sway}}{l_h}\dot{\theta}_l + \frac{g}{l_h}\theta_l = 0$$

Comparando con $s^2 + 2\zeta\omega_n s + \omega_n^2$, con $\omega_n = \sqrt{g/l_h}$:

$$\boxed{K_{sway}(l_h) = 2\,\zeta\,\sqrt{g\,l_h(t)}}$$



Gain scheduling: K_{sway} se ajusta con la longitud del cable l_h.

En la práctica

El amortiguamiento crítico ($\zeta = 1$) sobrecorrige la posición; se usa $\zeta = 0{,}5$ (subamortiguado) para mantener la carga cerca del objetivo.

Modelo del carro simplificado

Sistema simplificado

$$\begin{cases} \dot{\theta}_{tm}(t) = \omega_{tm}(t) \\ \dot{\omega}_{tm}(t) = -\frac{b_{t,eq}}{J_{t,eq}}\omega_{tm}(t) + \frac{1}{J_{t,eq}} T^*_{tm}(t) \end{cases}$$

Forma matricial: $\dot{\mathbf{x}} = A\mathbf{x} + B_c u$; $y = C\mathbf{x}$

$$\begin{bmatrix} \dot{\theta}_{tm}(t) \\ \dot{\omega}_{tm}(t) \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{b_{t,eq}}{J_{t,eq}} \end{bmatrix} \begin{bmatrix} \theta_{tm}(t) \\ \omega_{tm}(t) \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{J_{t,eq}} \end{bmatrix} T^*_{tm}(t)$$

$$y(t) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \theta_{tm}(t) \\ \omega_{tm}(t) \end{bmatrix} = \theta_{tm}(t)$$

Observador de estado

Ecuación del observador

$$\dot{\hat{\mathbf{x}}} = (A - K_e C)\hat{\mathbf{x}} + B_c u + K_e y$$

con matriz de ganancia $K_e = \begin{bmatrix} k_{\theta t} \\ k_{\omega t} \end{bmatrix}$

Acción integral

$$\dot{x}_I(t) = y(t) - \tilde{y}(t) = \theta_{tm}(t) - \tilde{\theta}_{tm}(t)$$

Esto se inyecta como control adicional en la dinámica estimada

Sistema aumentado del observador

Estado ampliado

$$\tilde{\mathbf{x}}_{\text{aug}}(t) = \begin{bmatrix} \tilde{\theta}_{tm}(t) \\ \tilde{\omega}_{tm}(t) \\ x_I(t) \end{bmatrix}$$

Forma matricial del observador aumentado

$$\begin{bmatrix} \dot{\tilde{\theta}}_{tm}(t) \\ \dot{\tilde{\omega}}_{tm}(t) \\ \dot{x}_I(t) \end{bmatrix} = \begin{bmatrix} -k_{\theta t} & 1 & 0 \\ -k_{\omega t} & -\frac{b_{t,eq}}{J_{t,eq}} & k_{it} \\ -1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \tilde{\theta}_{tm}(t) \\ \tilde{\omega}_{tm}(t) \\ x_I(t) \end{bmatrix} + \begin{bmatrix} 0 \\ \frac{1}{J_{t,eq}} \\ 0 \end{bmatrix} T^*_{tm}(t) + \begin{bmatrix} k_{\theta t} \\ k_{\omega t} \\ 1 \end{bmatrix} \theta_{tm}(t)$$

Salida estimada

$$\tilde{y}(t) = \tilde{\theta}_{tm}(t)$$

Asignación de polos del observador

Polinomio característico aumentado

$$p(s) = \det(sI - A_{\text{aug}}) = s^3 + s^2\left(k_{\theta t} + \frac{b_{t,eq}}{J_{t,eq}}\right) + s\left(k_{\omega t} + k_{\theta t}\frac{b_{t,eq}}{J_{t,eq}}\right) + k_{it}$$

Polinomio deseado

Tres polos coincidentes en $\omega_{\text{obs},t} = -20\frac{b_{t,eq}}{J_{t,eq}} = -14{,}4048\, rad/s$:

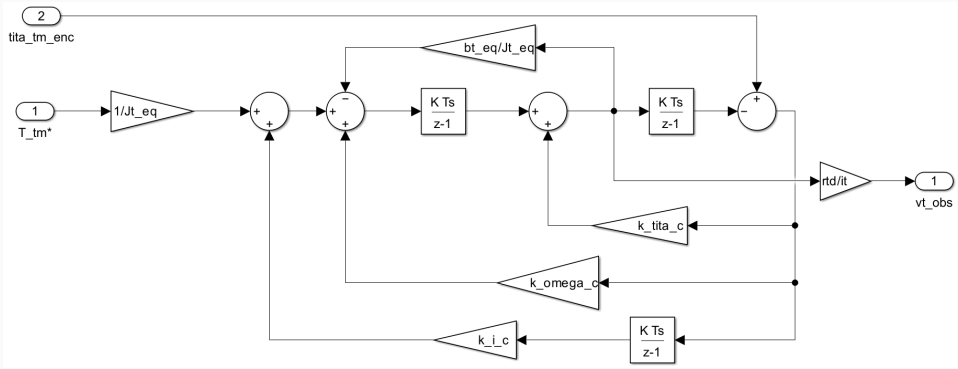
$$p_d(s) = (s + \omega_{\text{obs},t})^3 = s^3 + 3\,\omega_{\text{obs},t}\, s^2 + 3\,\omega_{\text{obs},t}^2\, s + \omega_{\text{obs},t}^3$$

Igualación de coeficientes

$$k_{\theta t} = 3\,\omega_{\text{obs},t} - \frac{b_{t,eq}}{J_{t,eq}}, \quad k_{\omega t} = 3\,\omega_{\text{obs},t}^2 - k_{\theta t}\frac{b_{t,eq}}{J_{t,eq}}, \quad k_{it} = \omega_{\text{obs},t}^3$$

Diagrama observador PI del carro

$$\begin{cases} \dot{\tilde{\theta}}_{tm}(t) = \tilde{\omega}_{tm}(t) + k_{\theta t}\left(\theta_{tm}(t) - \tilde{\theta}_{tm}(t)\right) \\ \dot{\tilde{\omega}}_{tm}(t) = \frac{1}{J_{t,eq}} T^*_{tm}(t) + k_{\omega t}\left(\theta_{tm}(t) - \tilde{\theta}_{tm}(t)\right) + k_{it}\, x_I(t) \\ \dot{x}_I(t) = \theta_{tm}(t) - \tilde{\theta}_{tm}(t) \end{cases}$$



Modelo de izaje para control de velocidad

Balance de torques en el eje del motor

Agrupando las dinámicas externas en un torque de perturbación T_{pert}:

$$J_{h,eq}\,\frac{d\omega_{hm}}{dt} + b_{h,eq}\,\omega_{hm} = T_{hm} - T_{pert}$$

$$T_{pert} = \frac{F_{hw}\,r_{hd}}{i_h}$$

Función de transferencia de la planta

$$G_{vel}(s) = \frac{\Omega_{hm}(s)}{T_{hm}(s)} = \frac{1}{J_{h,eq}\,s + b_{h,eq}}$$

Casi un integrador

Fricción muy baja $\Rightarrow$ polo $P_{mec} = -b_{h,eq}/J_{h,eq} \approx 0$: la planta se comporta casi como un **integrador puro**.

Diseño del lazo regulador (PI)

Controlador y lazo cerrado

$C_{vel}(s) = K_p + \dfrac{K_i}{s}$, que da:

$$T(s) = \frac{K_p\, s + K_i}{J_{h,eq}\, s^2 + (b_{h,eq} + K_p)\, s + K_i}$$

Asignación de polos ($s^2 + 2\zeta\omega_n s + \omega_n^2$)

$$K_p = 2\,\zeta\,\omega_n\, J_{h,eq} + b_{h,eq} \quad ; \quad K_i = \omega_n^2\, J_{h,eq}$$

Con $\omega_n \approx 8$ rad/s (rápido sin excitar la resonancia del cable) y $\zeta = 1$ (sin sobrepaso).

Rigidez

Esta sintonía da la "rigidez" necesaria para **sostener la carga** suspendida.

Compensación de gravedad (feedforward)

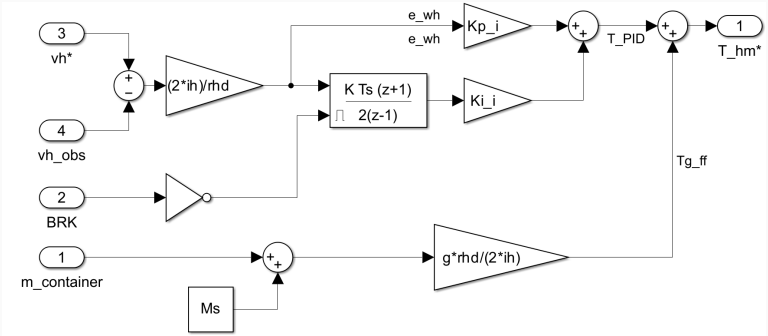
Pre-alimentación del peso estático

$$T_{g,ff} = \frac{m_l(TLK)\, g\, r_{hd}}{2\, i_h}$$

Ley de control del lazo: $T^*_{hm}(k) = T_{PI}(k) + T_{g,ff}$

Ventaja

El término integral sólo corrige **errores de modelo**; el peso lo sostiene el feedforward $\Rightarrow$ respuesta más rápida y sin caída al arrancar.



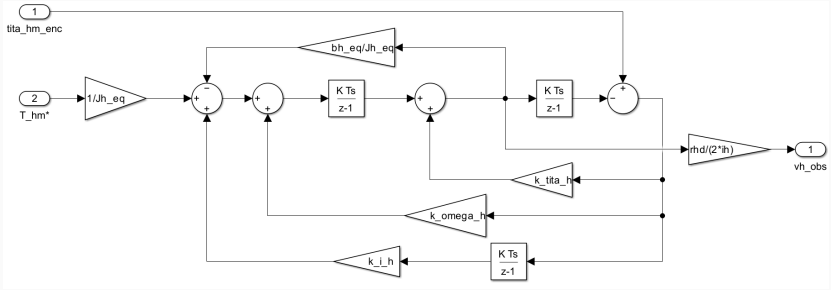
Anti-windup y observador del izaje

Anti-windup (dos frenos)

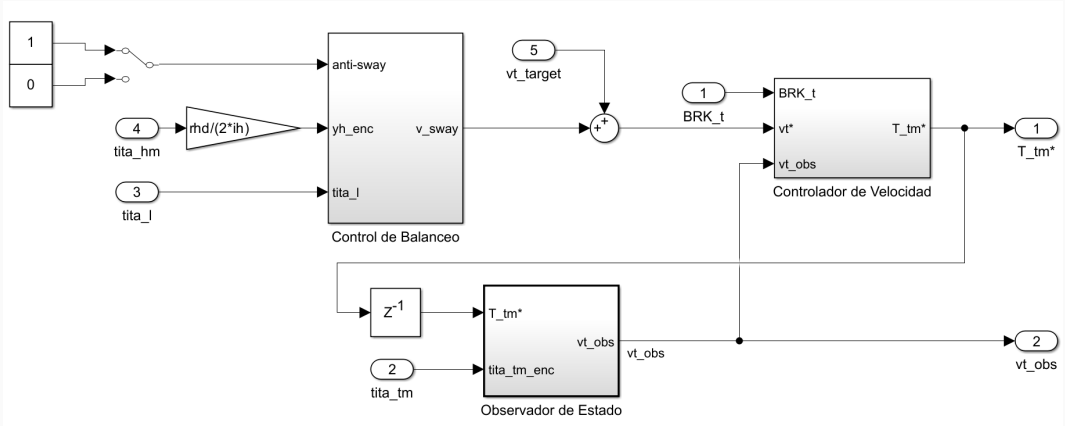
El izaje tiene freno de operación BRK_h y de emergencia BRK_{hE}: **cualquiera** de los dos resetea el integrador (igual criterio que el carro).

Observador con acción integral

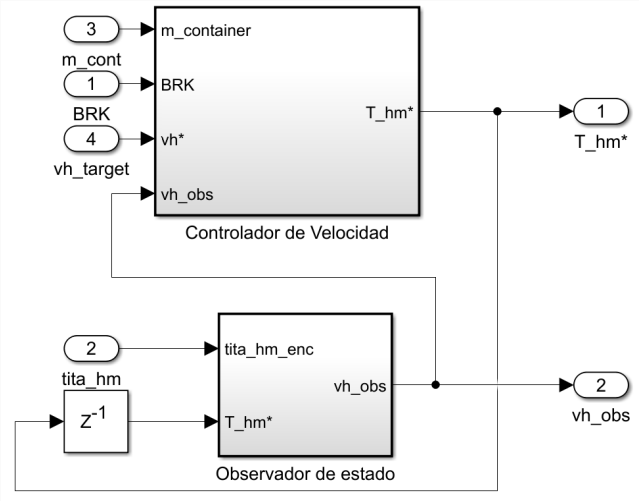
Análogo al del carro, con las matrices del izaje. Ganancias por **separación de anchos de banda**: $\omega_{obs\,h} = 20\times$ el polo dominante de la planta, siendo $\omega_{obs\,h} = -9{,}5197\ rad/s$



Controlador regulatorio del carro (Simulink)



Controlador regulatorio del izaje (Simulink)



Nivel 1 — Control supervisor (DEDS)

Nivel 1: el cerebro lógico de la grúa

Autómata de **eventos discretos** (Stateflow), $T_{s1} = 20$ ms. Organizado en **procesos paralelos** que se evalúan simultáneamente:

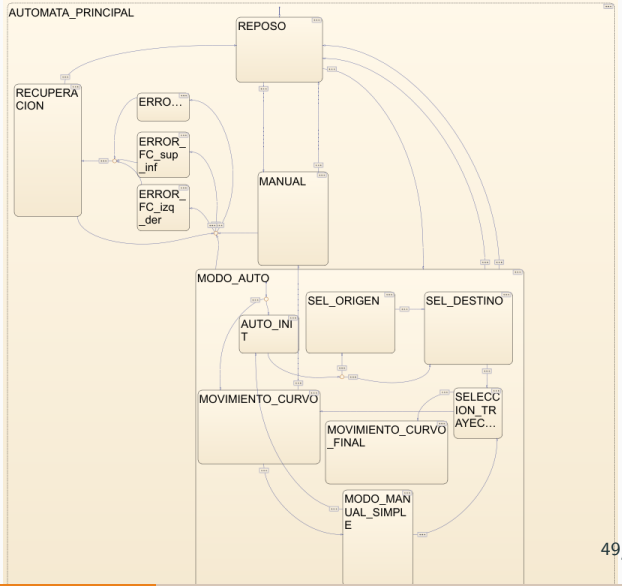
- **Autómata principal** de modos.
- Gestión de **twistlocks**.
- Análisis espacial: **perfil e índices** (LIDAR).
- **Suavizado** de consignas (jerk/acel.).
- **Límites suaves** de velocidad.
- **Watchdog** de comunicación.

Entradas / Salidas

Sensores + joystick + fallas $\rightarrow$ consignas vx_cmd/vy_cmd, frenos *BRK*, TLK, vh_lim, heartbeat.

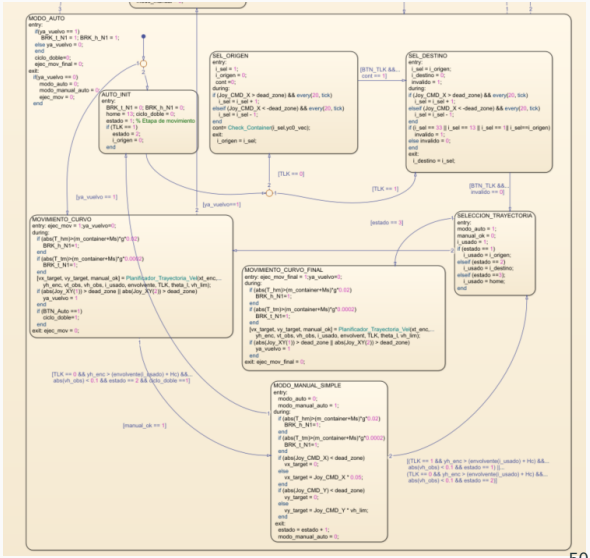
Autómata principal de modos de operación

- **Reposo**: detenido, frenos cerrados.
- **Manual**: control directo por joystick.
- **Automático**: ciclo autónomo de carga/descarga.
- **Recuperación**: resguardo tras falla.
- **Errores**: por tipo de fin de carrera.



Modo automático: ciclo coordinado sin colisiones

- Secuencia **origen** → **destino** gestionada por el estado del ciclo y los índices seleccionados.
- Una **función planificadora** genera las consignas de velocidad (vx_target, vy_target) coordinando ambos ejes.
- Usa la **envolvente de seguridad** del perfil para **evadir colisiones** con la pila de contenedores.
- Movimientos **simultáneos** (izaje + traslación) para optimizar el tiempo de ciclo.
- **Transferencia suave** a manual si el operador interviene (*bumpless*).

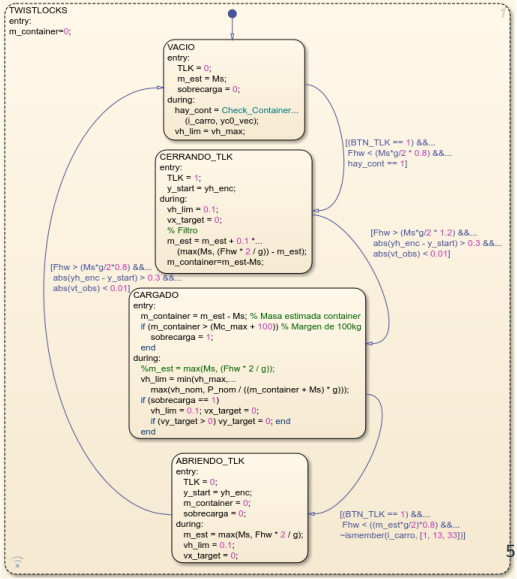


Twistlocks

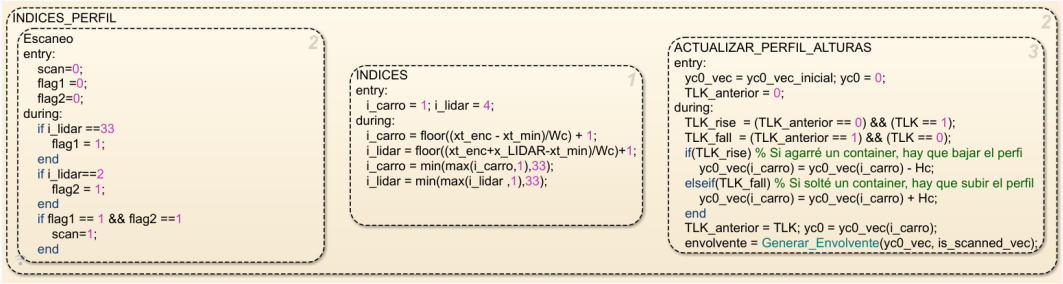
Cuatro estados: `VACIO` → `CERRANDO` → `CARGADO` → `ABRIENDO`.

- Toma/libera la carga sólo con el spreader **apoyado** (interlock).
- **Estima la masa** del contenedor con la celda de carga.
- Detecta **sobrecarga** y limita la velocidad por curva de potencia.

$$v_{h,lim}(m_l) = \begin{cases} v_{h\,\text{máx}} & \text{si } m_l < 32500kg \\ \frac{P_{h_nom}}{m_l\, g} & \text{si } m_l \geq 32500kg \end{cases}$$



Análisis espacial



Límites suaves de velocidad

```
VELOCIDAD_LIMITES
during:
    % LÍMITE INFERIOR DE IZAJE
    if (TLK == 1)
        dist = yh_enc - (yc0 + 0.5 + Hc); T_retardo = 0.8;
    else
        dist = yh_enc - (yc0 + 0.5); T_retardo = 0.4;
    end
    % Compensación con retardo dinámico
    dist_efectiva = dist - (abs(vy_suave) * T_retardo);
    if (dist_efectiva < 0)
        dist_efectiva = 0;
    end
    acc_plan = ah_max * 0.5;
    v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
    if (v_segura > vh_lim)
        v_segura = vh_lim;
    end
    % Si pide moverse más rápido, se recorta
    if (vy_suave < -v_segura)
        vy_suave = -v_segura;
    end
    % LÍMITE SUPERIOR DE IZAJE
    dist = (yh_max - 2) - yh_enc;
    dist_efectiva = dist - (abs(vy_suave) * T_retardo);
    if (dist_efectiva < 0)
        dist_efectiva = 0;
    end
    v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
    if (v_segura > vh_lim)
        v_segura = vh_lim;
    end
    if (vy_suave > v_segura)
        vy_suave = v_segura;
    end
```

```
    % LÍMITE DERECHO DEL CARRO
    T_retardo = 5; % Retardo mayor para el carro
    dist = (xt_max - 2) - xt_enc;
    dist_efectiva = dist - (abs(vx_suave) * T_retardo);
    if (dist_efectiva < 0)
        dist_efectiva = 0;
    end
    acc_plan = at_max * 0.5;
    v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
    if (vx_suave > v_segura)
        vx_suave = v_segura;
    end
    % LÍMITE IZQUIERDO DEL CARRO
    dist = xt_enc - (xt_min + 2);
    dist_efectiva = dist - (abs(vx_suave) * T_retardo);
    if (dist_efectiva < 0)
        dist_efectiva = 0;
    end
    v_segura = sqrt(0.01 + 2 * acc_plan * dist_efectiva);
    if (vx_suave < -v_segura)
        vx_suave = -v_segura;
    end
```

Suavizado y watchdog

Suavizado

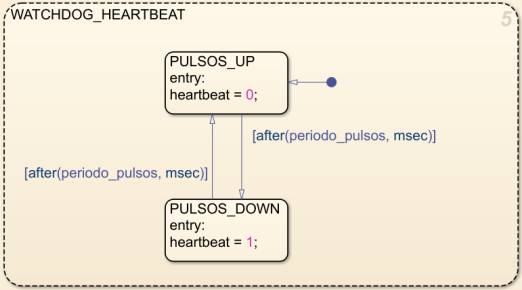
Convierte consignas crudas en v_{suave}.

```
SUAVIZADO_VELOCIDAD                                    4
entry:
  ax_act = 0; ay_act = 0;
  jerk_max = 1.5;
  vx_suave = 0; vy_suave = 0;
  vx_target = 0; vy_target = 0;
during:
  [vx_suave, ax_act] = Suavizador_Cinematico...
      (vx_target, vx_suave, ax_act, at_max, jerk_max);
  [vy_suave, ay_act] = Suavizador_Cinematico...
      (vy_target, vy_suave, ay_act, ah_max, jerk_max);
```

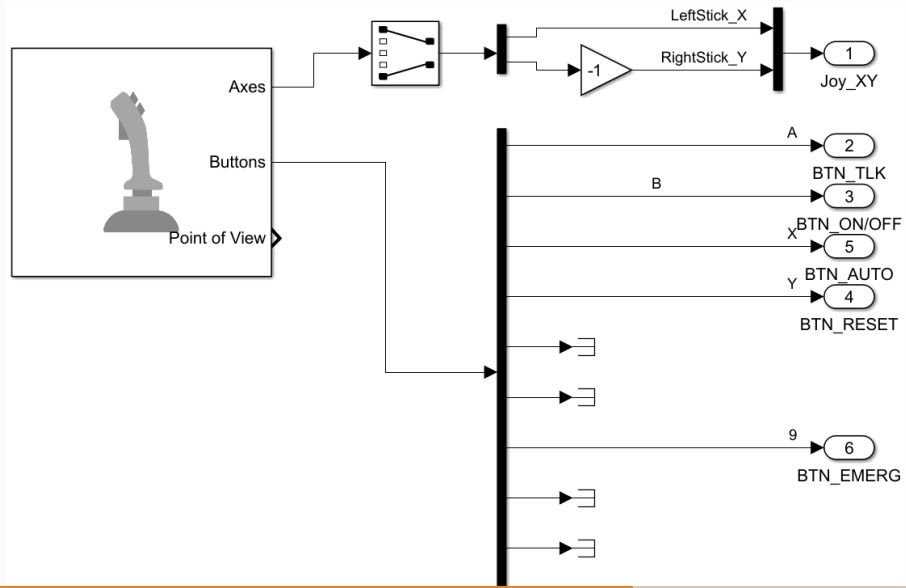
```
MATLAB Function
[v_out, a_out] = Suavizador_Cinematico...
   (v_target, v_act, a_act, A_MAX, J_MAX)
```

Watchdog

Genera un heartbeat cada cierto tiempo que recibe el Nivel 0.



Control del joystick



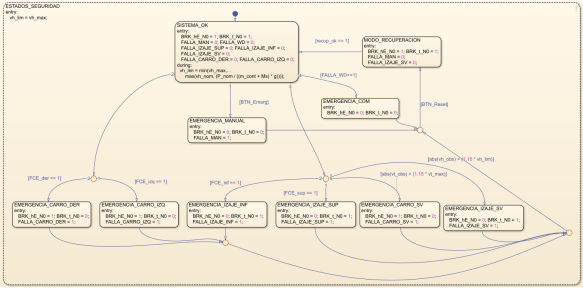
Nivel 0 — Protección

Nivel 0

- Capa de protección más baja, pero con mayor jerarquía.
- Salvaguarda la integridad de la grúa, la carga y el personal.
- Autoridad para sobrescribir cualquier comando del Nivel 1, cortando energía y aplicando frenos.
- Supervisa la planta de forma **permanente** (redundancia de sensores).

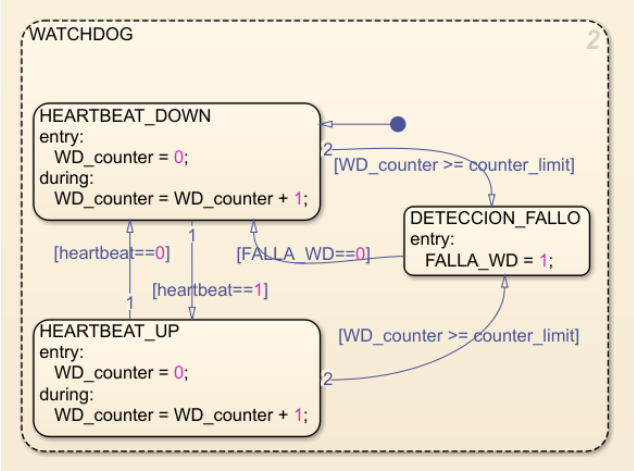
Autómata de seguridad: frenado selectivo

- Estado nominal `SISTEMA_OK` (ambos frenos abiertos).
- **Frenado selectivo**: sólo se detiene el **eje afectado**.
  - Falla izaje ⇒ frena izaje, carro libre.
  - Falla carro ⇒ frena carro, izaje libre.
  - Emergencia manual ⇒ ambos.
- Rearme manual → `RECUPERACION`.



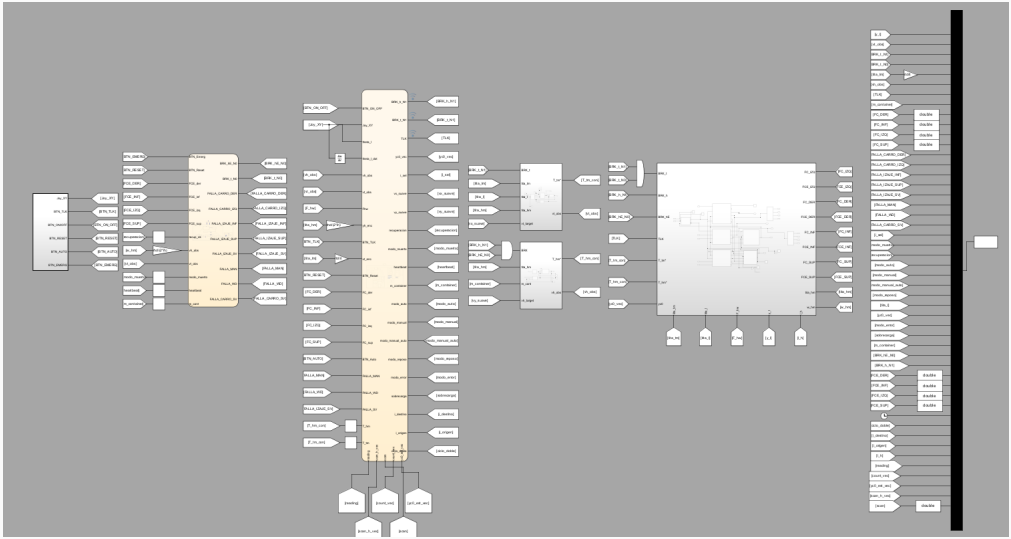
Monitor de comunicaciones

- Persigue el **heartbeat** del supervisor.
- Si el contador supera el límite seguro $FALLA_{WD}$.
- Dispara la parada de emergencia.



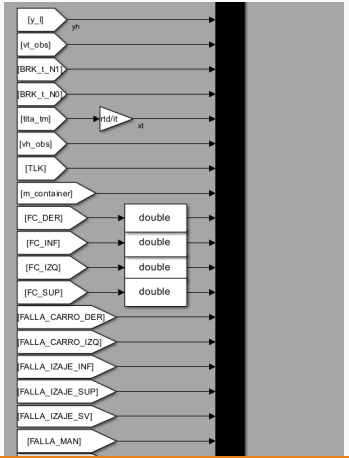
Comunicación y Animación con Python

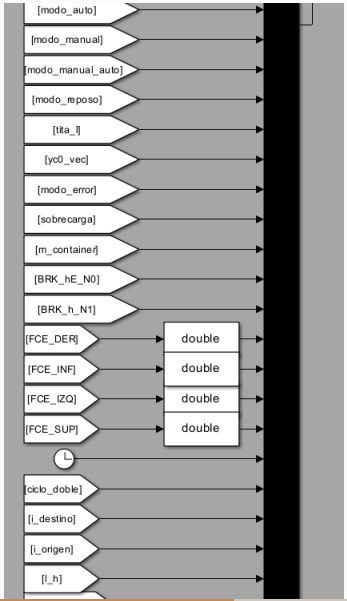
Integración completa en Simulink



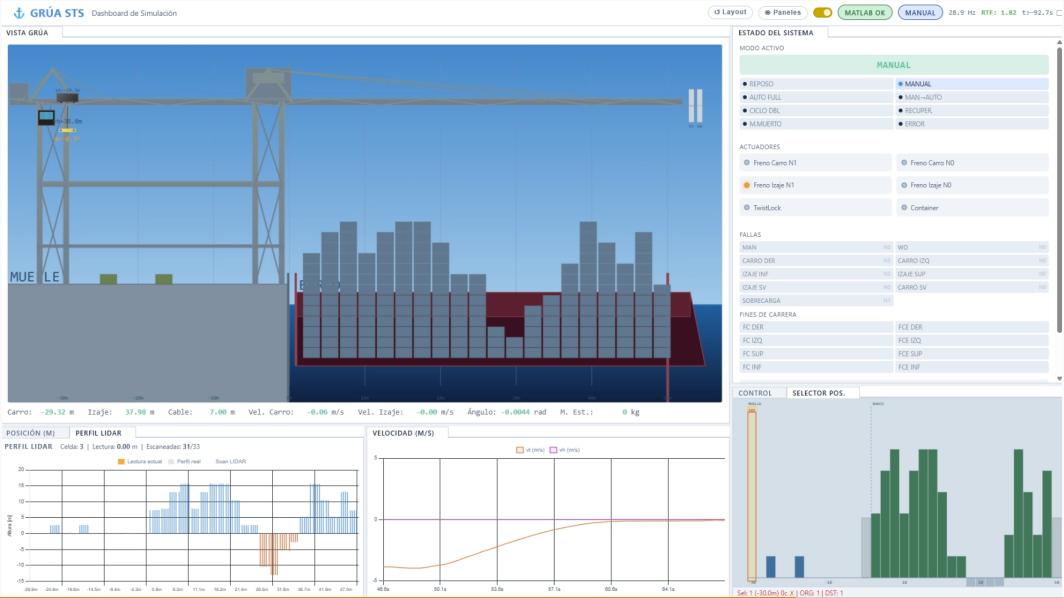
Planta física + Nivel 2 (control) + Nivel 1 (supervisor) + Nivel 0 (protección) + UDP

Comunicación UDP: Simulink Python

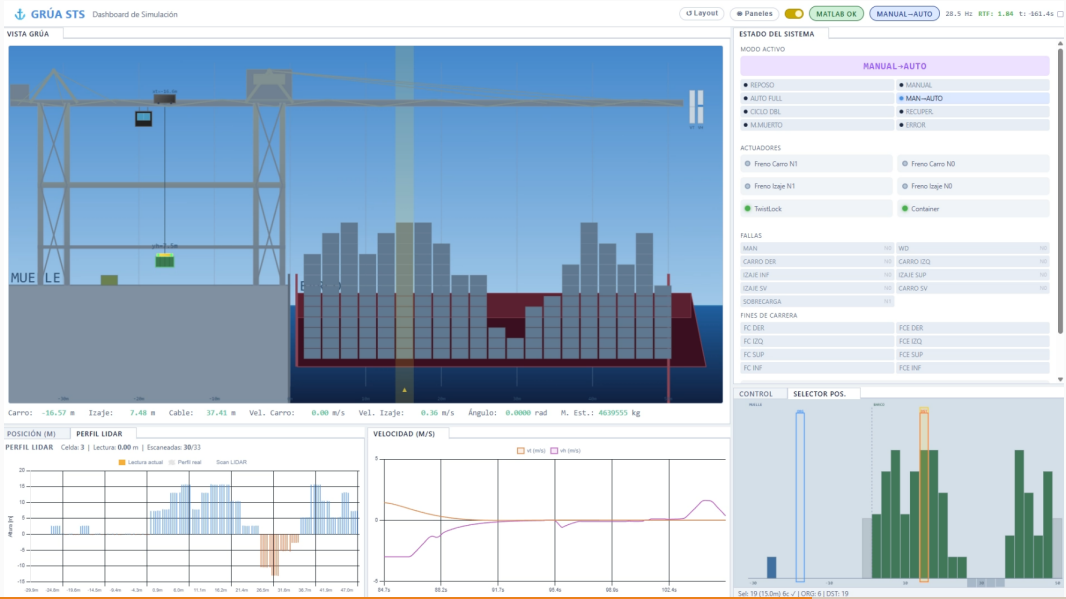




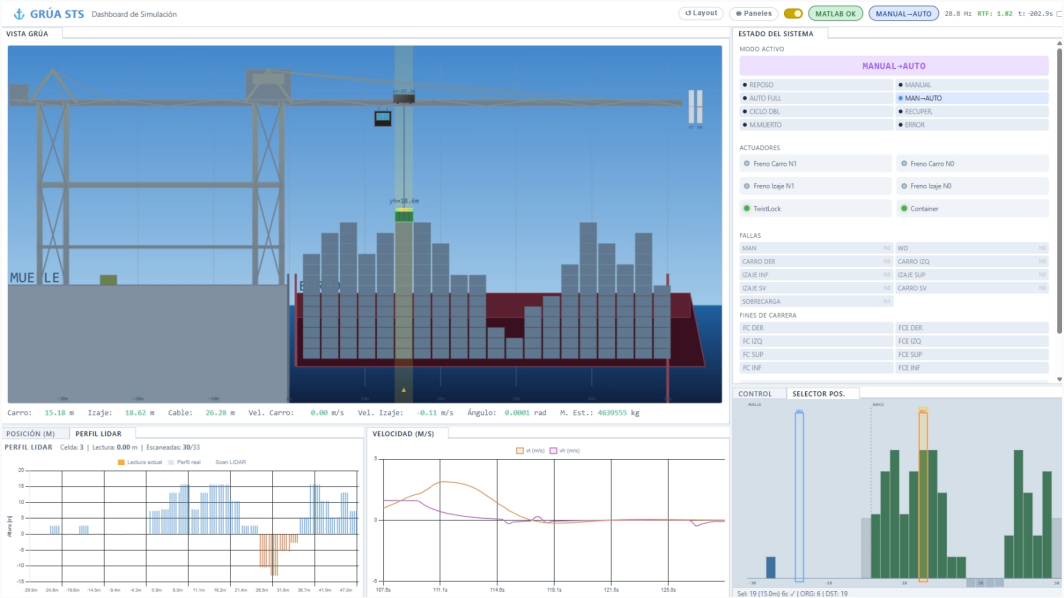
Modo manual


GRÚA STS
Dashboard de Simulación
Layout
Paneles
MATLAB OK
MANUAL
28.9 Hz RTF: 1.82 t=92.7s
VISTA GRÚA
ESTADO DEL SISTEMA
MODO ACTIVO
MANUAL
REPOSO
MANUAL
AUTO FULL
MAN→AUTO
CICLO DBL
RECUPER.
M.MUERTO
ERROR
ACTUADORES
Freno Carro N1
Freno Carro N0
Freno Izaje N1
Freno Izaje N0
TwistLock
Container
FALLAS
MAN
WD
CARRO DER
CARRO IZQ
IZAJE INF
IZAJE SUP
IZAJE SV
CARRO SV
SOBRECARGA
FINES DE CARRERA
FC DER
FCE DER
FC IZQ
FCE IZQ
FC SUP
FCE SUP
FC INF
FCE INF
MUE LE
Carro: -29.32 m Izaje: 37.98 m Cable: 7.00 m Vel. Carro: -0.06 m/s Vel. Izaje: -0.00 m/s Ángulo: -0.0044 rad M. Est.: 0 kg
POSICIÓN (M)
PERFIL LIDAR
PERFIL LIDAR Celda: 3 | Lectura: 0.00 m | Escaneadas: 31/33
Lectura actual
Perfil real
Scan LIDAR
Altura (m)
VELOCIDAD (M/S)
vt (m/s)
vh (m/s)
CONTROL
SELECTOR POS.
Sel: 1 (-30.0m) 0c X | ORG: 1 | DST: 1

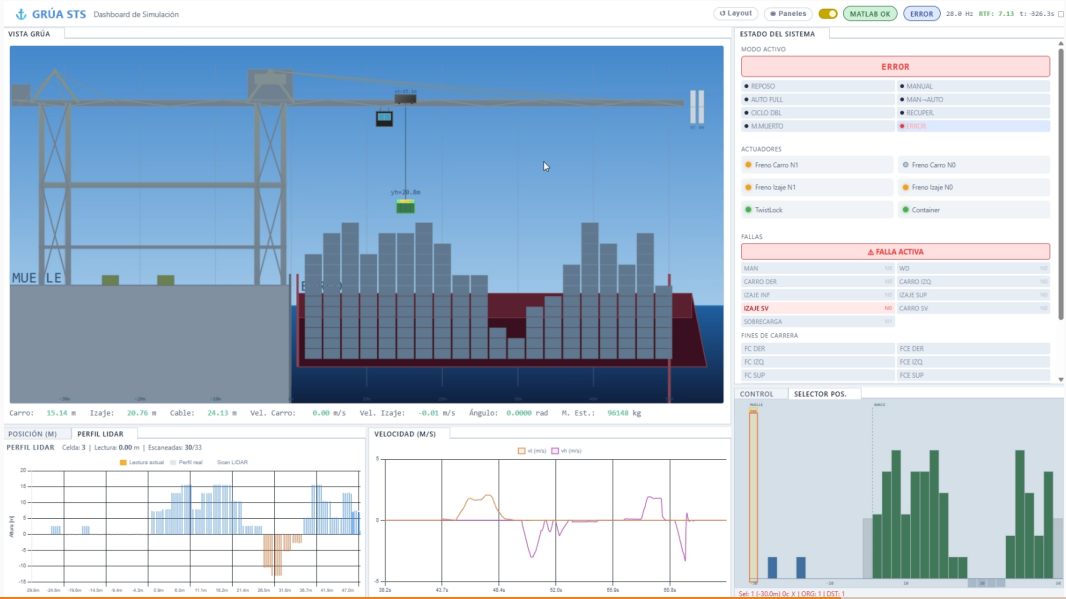
Modo automático: seguimiento de trayectoria



Liberación de la carga



Estado de error



Implementación en CODESYS

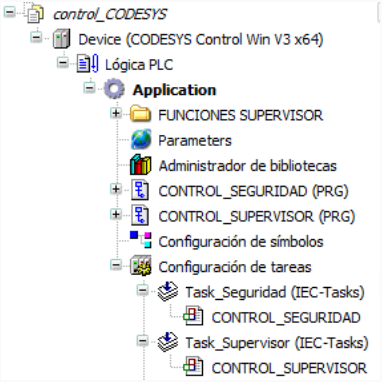
CODESYS: codificación manual (IEC 61131)

De Stateflow/Simulink a un PLC

Los Niveles 0 y 1 se traducen a **SFC** (Sequential Function Charts) con cálculos en **ST** (Structured Text); el Nivel 2 se rehace con bloques de función de Matlab (*simil ST*).

Ejecución

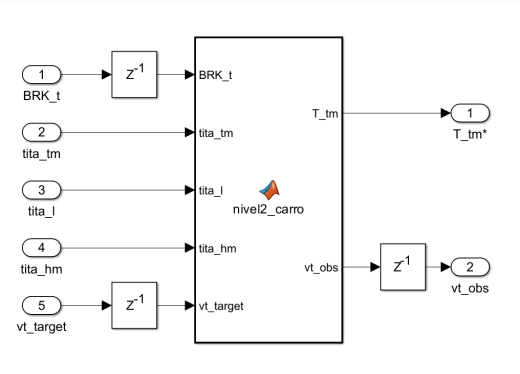
Dos POUs principales (Niveles 0 y 1) cíclicos e independientes. En el PLC virtual (una sola CPU) se ejecutan secuencialmente intercalados, equivalente a paralelo.



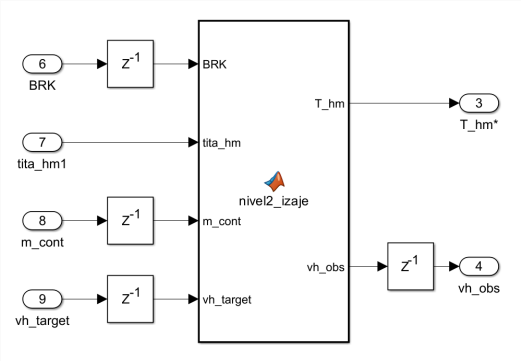
Árbol de componentes del proyecto

Control regulatorio en simil ST: diagramas

Se rehace el Nivel 2 con la misma lógica pero con bloques de función de Matlab (simil ST); se agregan bloques *Delay* para resolver loops algebraicos.



(a) Control del carro



(b) Control del izaje

Control regulatorio en simil ST: código

El código de cada controlador (dentro de cada bloque de función de Matlab):

```
%% 1. Anti-sway
yh      = double(tita_hm) * (rhd / (2 * ih));
h       = max(Yt0 - yh, 0.5);
v_sway = double(tita_l) * 2 * zitta * sqrt(g * h);
```

```
%% 2. Corrección de referencia
vt_cmd    = double(vt_target) + v_sway;
vt_sat    = max(min(vt_cmd, vt_max), -vt_max);
delta     = vt_sat - rl_state;
rl_state = rl_state + max(min(delta, at_max * Ts2), -at_max * Ts2);
vt_ref    = rl_state;
```

```
%% 3. Observador de Luenberger
tita_delayed = tita_prev;
tita_prev    = double(tita_tm);

err_pos    = tita_delayed - theta_hat;
v_est      = omega_hat + k_tita_c * err_pos;
omega_dot = T_prev / Jt_eq + k_i_c * x3 + k_omega_c * err_pos;

theta_hat = theta_hat + Ts2 * v_est;
omega_hat = omega_hat + Ts2 * omega_dot;
x3         = x3         + Ts2 * err_pos;

vt_out = v_est * (rtd / it);
```

```
%% 4. PI trapezoidal con reset anti-windup
err = vt_ref - vt_out;
if ~double(BRK_t)
    int_e = 0;
else
    int_e = int_e + (Ts2 / 2) * (err + prev_e);
end
prev_e = err;
```

```
%% 1. Modulación de referencia
vh_sat    = max(min(double(vh_target), vh_max), -vh_max);
delta     = vh_sat - rl_state;
rl_state = rl_state + max(min(delta, ah_max * Ts2), -ah_max * Ts2);
vh_ref    = rl_state;
```

```
%% 2. Observador de Luenberger
tita     = double(tita_hm);
err_pos = tita - theta_hat;

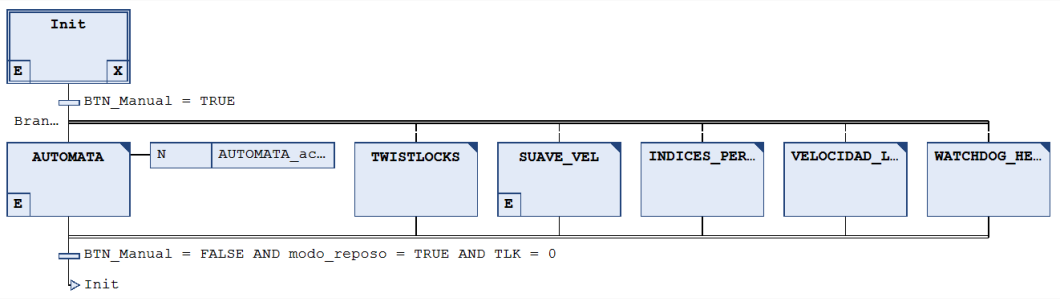
v_est      = omega_hat + k_tita_h * err_pos;
omega_dot = -(bh_eq / Jh_eq) * v_est + T_prev / Jh_eq ...
            + k_i_h * x3 + k_omega_h * err_pos;

theta_hat = theta_hat + Ts2 * v_est;
omega_hat = omega_hat + Ts2 * omega_dot;
x3         = x3         + Ts2 * err_pos;

vh_out = v_est * (rhd / (2 * ih));
```

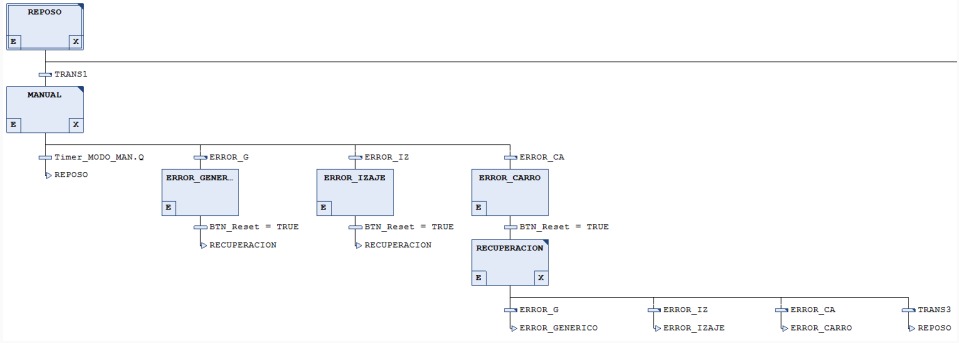
```
%% 3. PI trapezoidal con reset anti-windup
err = vh_ref - vh_out;
if ~double(BRK)
    int_e = 0;
else
    int_e = int_e + (Ts2 / 2) * (err + prev_e);
end
prev_e = err;

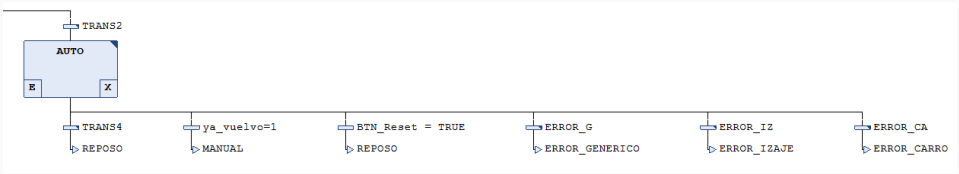
m_total = Ms + double(m_cont);
```



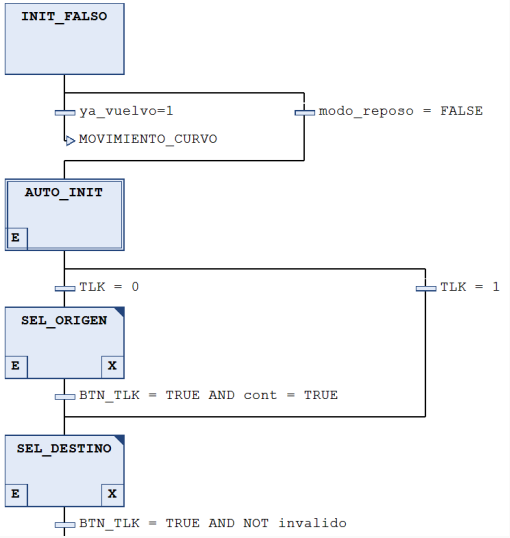
Conversión de bloques Stateflow a SFC (Sequential Function Chart) manteniendo estados y transiciones

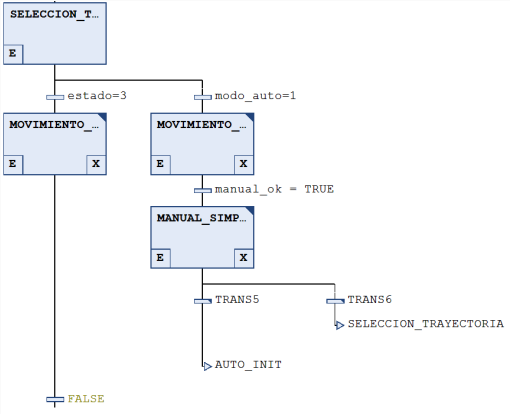
Autómata principal en CODESYS



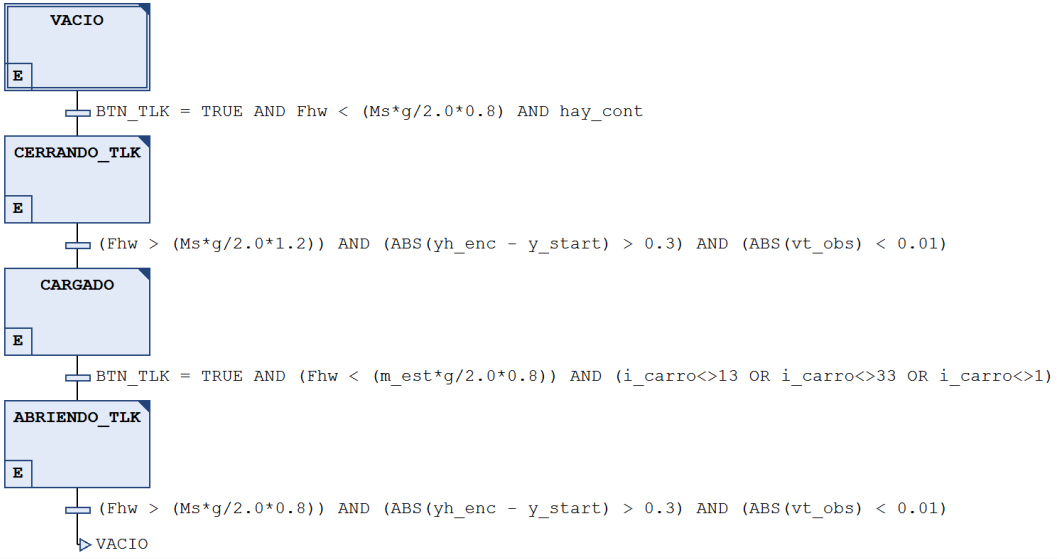


Modo automático

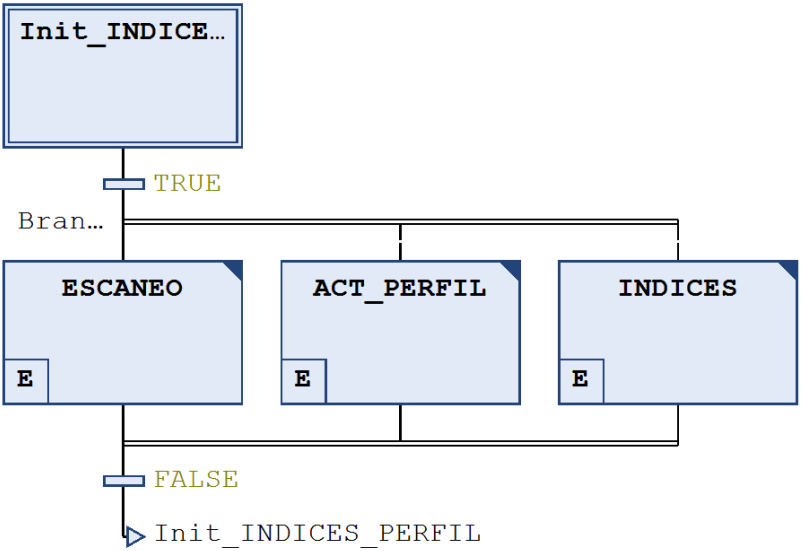




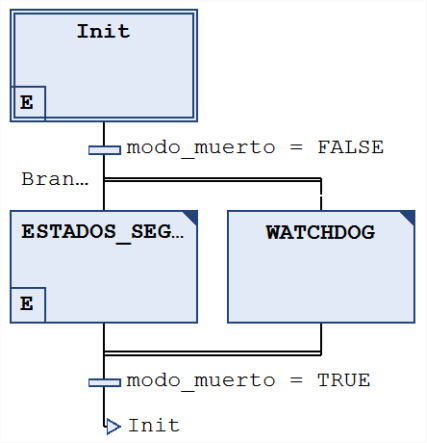
Twistlocks

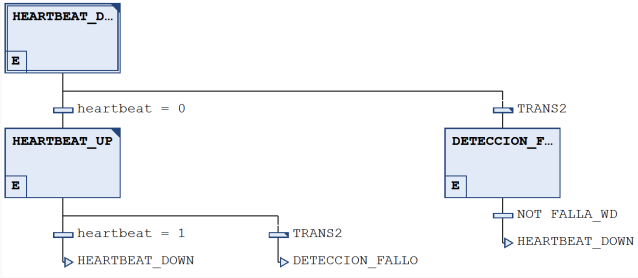

VACIO
E
BTN_TLK = TRUE AND Fhw < (Ms*g/2.0*0.8) AND hay_cont
CERRANDO_TLK
E
(Fhw > (Ms*g/2.0*1.2)) AND (ABS(yh_enc - y_start) > 0.3) AND (ABS(vt_obs) < 0.01)
CARGADO
E
BTN_TLK = TRUE AND (Fhw < (m_est*g/2.0*0.8)) AND (i_carro<>13 OR i_carro<>33 OR i_carro<>1)
ABRIENDO_TLK
E
(Fhw > (Ms*g/2.0*0.8)) AND (ABS(yh_enc - y_start) > 0.3) AND (ABS(vt_obs) < 0.01)
VACIO

Análisis espacial

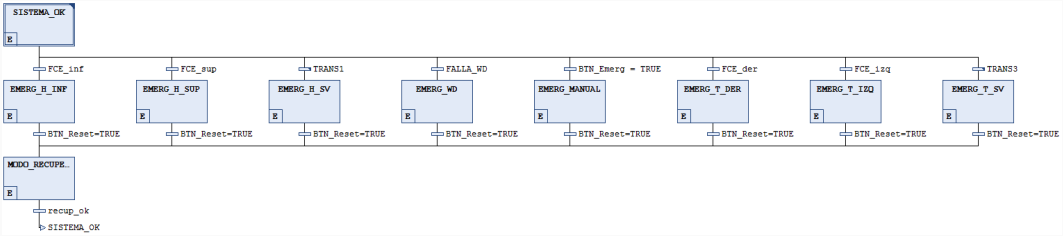


Nivel 0 CODESYS





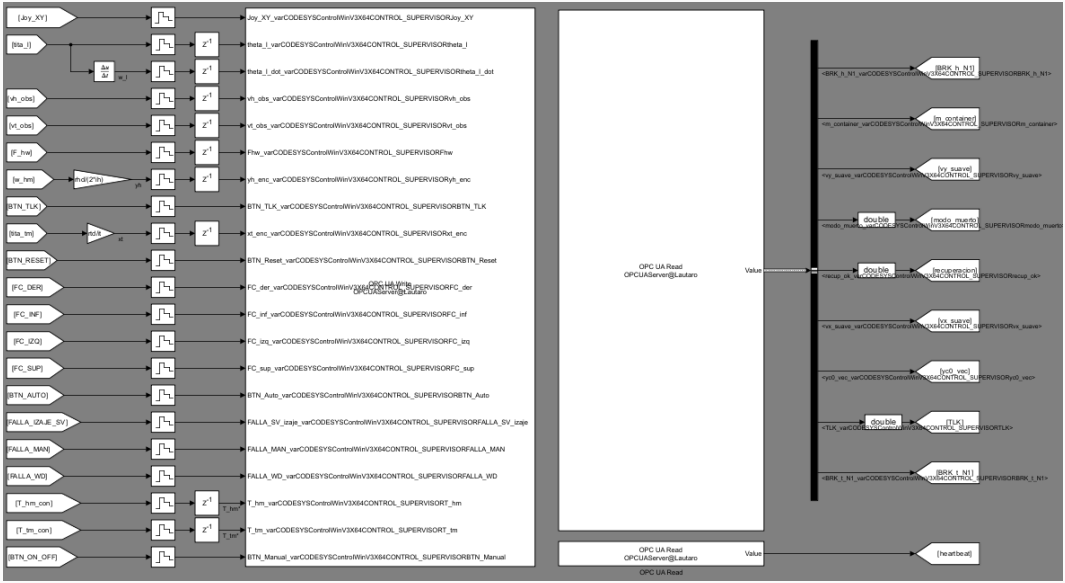
Autómata de protección



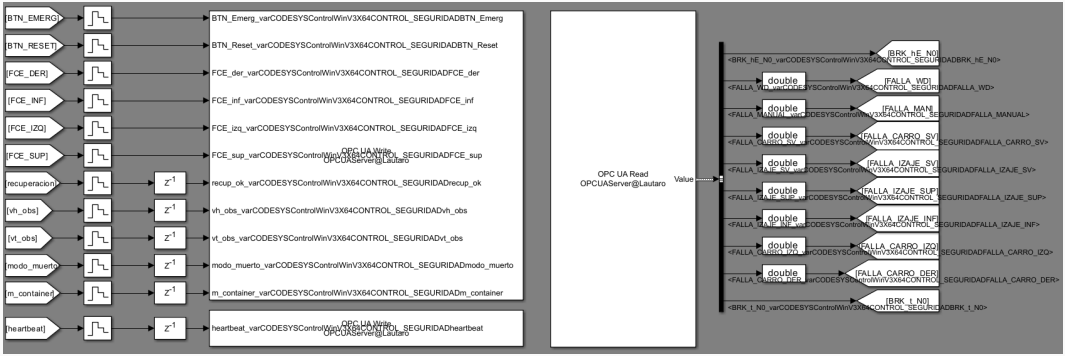
Comunicación OPC-UA Simulink/CODESYS

- CODESYS V3.5 SP21 Patch 2: servidor OPC-UA integrado, permite exponer variables del programa
- Matlab R2025b: cliente OPC-UA, permite conectar al servidor de CODESYS y leer/escribir variables en tiempo real
- Ajuste de seguridad: *User allow anonymous* habilitado para permitir conexión sin autenticación

Protocolo OPC-UA Nivel 1



Protocolo OPC-UA Nivel 0



Desafío: retardo de comunicación

Problema

OPC es protocolo TCP/IP diseñado para SCADA (retardos $\geq$ 500 ms), no para control tiempo real

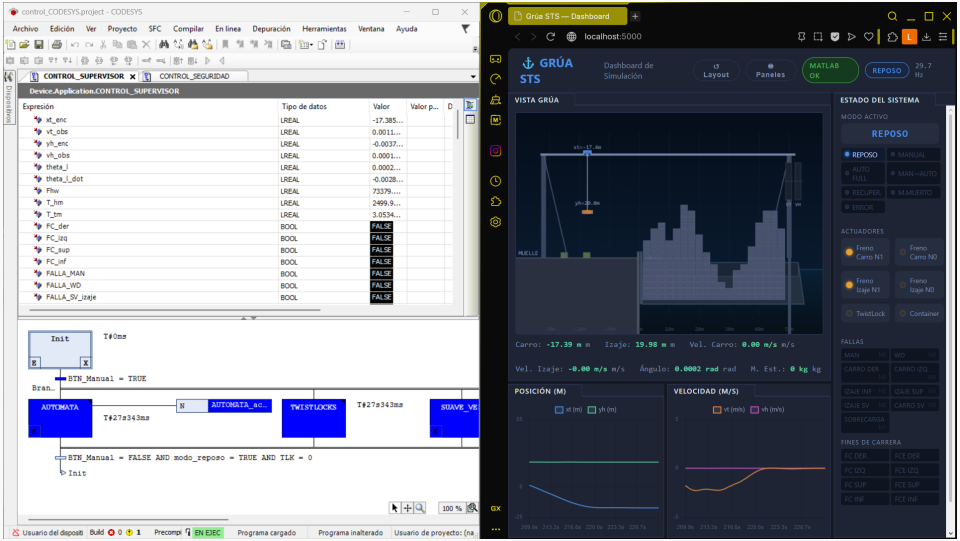
Solución implementada

Bloques de comunicación con tiempos de muestreo diferenciados:

- Control supervisor y protección: 500 ms
- Watchdog heartbeat: 20 ms (tiempo original)

Reduce carga en red permitiendo Simulink correr a 1s por segundo de simulación

Simulación con OPC-UA en funcionamiento



Generación automática de código ST

Generación automática de código ST

Herramienta: Simulink PLC Coder

Traduce automáticamente los Statecharts a **Texto Estructurado** (ST, norma IEC 61131-3).

Estructura del código generado

Autómata **plano y monolítico**: un `CASE` global sobre la variable de estado, con una densa red de `IF/ELSIF` anidados para resolver transiciones y acciones.

Manual (SFC) vs. autogenerado (ST)

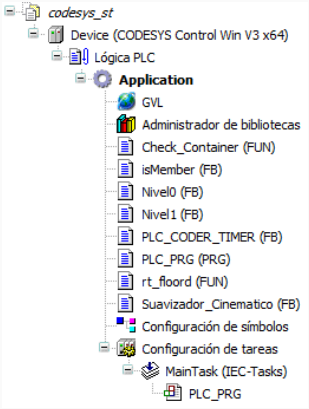
- **Eficiencia / CPU**: comparable (el compilador del PLC optimiza ambos).
- **Legibilidad y diagnóstico**: el ST autogenerado **carece de trazabilidad visual** $\Rightarrow$ difícil de interpretar en planta.

Por eso se optó por **SFC manual** para la arquitectura y ST sólo para los cálculos.

Fragmento de código ST autogenerado (PLC Coder)

```
CASE is_c3_Nivel OF
    Nivel_IN_Automata:
        (* During 'Automata': '<S1>:361' *)
        (* '<S1>:366:1' sf_internal_predicateOutput = 0 | (duration((BTN_ON_OFF==0 && modo_reposo == 1),sec)>2 && ...  *)
        (* '<S1>:366:2' duration(TLK==0,sec)>2); *)
        IF (B <> 0.0) OR (b_modo_reposo <> 1.0) THEN
            duration_counter_1_m := 0;
        END_IF;
        temp1 := FALSE;
        IF duration_counter_1_m > 100 THEN
            IF b_TLK <> 0.0 THEN
                duration_counter_2 := 0;
            END_IF;
            IF duration_counter_2 > 100 THEN
                (* Transition: '<S1>:366' *)
                (* Exit Internal 'Automata': '<S1>:361' *)
                (* Exit Internal 'AUTOMATA_PRINCIPAL': '<S1>:1' *)
                CASE is_AUTOMATA_PRINCIPAL OF
                    Nivel_IN_ERROR_FC_izq_der:
                        is_AUTOMATA_PRINCIPAL := Nivel_IN_NO_ACTIVE_CHILD;
                    Nivel_IN_ERROR_FC_sup_inf:
                        is_AUTOMATA_PRINCIPAL := Nivel_IN_NO_ACTIVE_CHILD;
                    Nivel_IN_ERROR_OTRO:
                        is_AUTOMATA_PRINCIPAL := Nivel_IN_NO_ACTIVE_CHILD;
                    Nivel_IN_MANUAL:
                        (* Exit 'MANUAL': '<S1>:8' *)
                        (* '<S1>:8:23' modo_manual = 0; *)
                        b_modo_manual := 0.0;
                        is_AUTOMATA_PRINCIPAL := Nivel_IN_NO_ACTIVE_CHILD;
                    Nivel_IN_MODO_AUTO:
                        (* Exit Internal 'MODO_AUTO': '<S1>:240' *)
                        CASE is_MODO_AUTO OF
                            Nivel_IN_AUTO_INIT:
                                is_MODO_AUTO := Nivel_IN_NO_ACTIVE_CHILD;
                            Nivel_IN_MODO_MANUAL_SIMPLE:
                                (* Exit 'MODO_MANUAL_SIMPLE': '<S1>:250' *)
                                (* '<S1>:250:23' estado = estado + 1; *)
                                estado := estado + 1.0;
                                (* '<S1>:250:24' modo_manual_auto = 0; *)
                                b_modo_manual_auto := 0.0;
                                is_MODO_AUTO := Nivel_IN_NO_ACTIVE_CHILD;
                            Nivel_IN_MOVIMIENTO_CURVO:
                                (* Exit 'MOVIMIENTO_CURVO': '<S1>:297' *)
                                (* '<S1>:297:18' ejec_mov = 0; *)
                                is_MODO_AUTO := Nivel_IN_NO_ACTIVE_CHILD;
                            c_Nivel_IN_MOVIMIENTO_CURVO:
                                (* Exit 'MOVIMIENTO_CURVO_FINAL': '<S1>:386' *)
                                (* '<S1>:386:15' ejec_mov_final = 0; *)
                                ejec_mov_final := 0.0;
                                is_MODO_AUTO := Nivel_IN_NO_ACTIVE_CHILD;
                            c_Nivel_IN_SELECCION_TRAYEC:
                                is_MODO_AUTO := Nivel_IN_NO_ACTIVE_CHILD;
```

Funcionamiento ST autogenerado




```
PROGRAM PLC_PRG
VAR
    // 1. Instancia del bloque generado
    ControlNivel1 : Nivel1;
    ControlNivel0 : Nivel0;

    // 2. ENTRADAS OPC UA
    opc_in_ON_OFF           : LREAL; // Botón ON/OFF
    opc_in_Joy_XY           : ARRAY [0..1] OF LREAL;
    opc_in_yh               : LREAL; // Posición cuerda izaje
    opc_in_xt               : LREAL; // Posición carro
    opc_in_FC_der           : BOOL;  // Fin de carrera derecho
    opc_in_FC_izq           : BOOL;  // Fin de carrera izquierdo
    opc_in_FC_sup           : BOOL;  // Fin de carrera superior
    opc_in_FC_INF           : BOOL;  // Fin de carrera inferior
    opc_in_T_hm             : LREAL; // Torque hoist motor
    opc_in_T_tm             : LREAL; // Torque trolley motor
    opc_in_tita_1           : LREAL;
    opc_in_tita_dl          : LREAL;
    opc_in_vh               : LREAL;
    opc_in_vt               : LREAL;
    opc_in_Fhw              : LREAL;
    opc_in_BTN_TLK          : BOOL;
```

```
ControlNivel1(
    ssMethodType := 1, // 1 = SS_STEP (siempre ejecutando)
    B := opc_in_ON_OFF,
    Joy_XY := opc_in_Joy_XY,
    yh := opc_in_yh,
    xt := opc_in_xt,
    FC_der := opc_in_FC_der,
    FC_izq := opc_in_FC_izq,
    FC_sup := opc_in_FC_sup,
    FC_INF := opc_in_FC_INF,
    T_hm := opc_in_T_hm,
    T_tm := opc_in_T_tm,
    tita_1 :=opc_in_tita_1,
    theta_l_dot := opc_in_tita_dl,
    vh_obs := opc_in_vh,
    vt_obs := opc_in_vt,
    Fhw := opc_in_Fhw,
    A := opc_in_BTN_TLK,
    b_Reset := opc_in_BTN_RESET,
    X := opc_in_BTN_AUTO,
    FALLA_WD := opc_in_FALLA_WD,
    FALLA_MAN := opc_in_FALLA_MAN,
    FALLA_IZAJE_SV := opc_in_FALLA_IZAJE_SV
);
```

Conclusiones

Conclusiones: logros por nivel

- **Nivel 2 — regulatorio:**
  - Seguimiento de velocidad preciso y **sin sobrepaso** en ambos ejes.
  - El izaje sostiene el peso **sin caída** de velocidad (compensación de gravedad).
  - El observador entrega una velocidad estimada limpia, sin ruido de derivar el encoder.
  - El *anti-sway* reduce el balanceo a valores despreciables al finalizar la maniobra $\Rightarrow$ apilado preciso.
- **Nivel 1 — supervisor:**
  - Ciclos completos de carga/descarga **autónomos**, sin colisionar con la pila (LIDAR).
  - La estimación de masa y el límite por curva de potencia **impiden la sobrecarga**.
  - El suavizado y los límites suaves logran trayectorias fluidas que nunca alcanzan los finales de carrera de forma abrupta.
- **Nivel 0 — protección:**
  - Ante las fallas inducidas (sobrevelocidad, FC de emergencia, parada manual, pérdida del *heartbeat*) lleva el sistema a un estado seguro.
  - Detiene **sólo el eje afectado**; la grúa nunca sigue operando con un comando desactualizado.

Validación e implementación

- Comportamiento validado en **simulación**, apoyada en una animación 2D en tiempo real que muestra la operación en los distintos modos.
- La lógica de control se trasladó a un PLC virtual en **CODESYS** (SFC + texto estructurado).
- Co-simulación con la planta vía **OPC-UA**: el controlador implementado **responde igual** que el modelo original.
- Objetivo de **productividad**: ≥ 30 contenedores por hora.

Trabajos futuros

- Llevar el control a **hardware real** sobre un PLC físico.
- Sustituir OPC-UA por un **enlace de menor latencia** apto para tiempo real.
- Profundizar la **planificación de trayectorias** y el control **anti-balanceo** para optimizar los tiempos de ciclo.
- Enriquecer el sensado del **perfil de obstáculos** para entornos más complejos.